LASSO / TECHNICAL GUIDE

How Lasso ranks files and results

A practical explanation of the algorithm and the program that runs it

Lasso retrieves real, versioned project sources, classifies their relevance, and orders them with a small amount of local personalization. This guide follows that process from the filesystem to the suggestion the user sees.

THE CENTRAL IDEA

Ranking has several ordered decisions. A direct match wins before popularity is considered. Document relationships and usage distinguish candidates only after the relevant tier and score band have been established.

Read this guide to understand	Pages
The data model and running program	2-3
How search and writing create candidates	4-6
The exact comparator, usage and diversity	7-10
UI policies and worked examples	11-13
Debugging, source map and formula appendix	14-16

Audience: readers familiar with functions, arrays, sorting, databases and basic mathematics. No information-retrieval or machine-learning background is required.

Implementation basis: the local working tree of Lasso on branch **codex/file-recommendation-ranking**, reviewed October 6, 2026. Constants are deterministic tuning choices and may change in later versions.

02 / IMPLEMENTATION GUIDE

Start with the data model

Lasso recommends both whole source files and objects inside those files. A spreadsheet can contribute hundreds of cells; a bibliography can contribute several separately citable papers. This distinction explains why relevance is computed per candidate while usage is usually shared at the file level.

Term	Meaning in the program
Project	A registered local directory and its history. Every recommendation and usage row is scoped to one project ID.
Stable file identity	**project_file_id** identifies a tracked file across snapshots and recognized renames. A path is the file name at one point in time.
Snapshot	An immutable version of the project. It records which stable files existed, their paths and their content hashes.
Scientific object	An addressable source item: cell, variable, row, code symbol, JSON value, paper, figure or similar indexed evidence.
Object version	The object in one snapshot, with its value, metadata and structural anchor such as a cell, line or reference-entry key.
Claim dependency	An accepted link from a span in a document to a particular source object and source snapshot.
Candidate	An eligible object or file prepared for ranking, carrying an ID, path, raw score and relevant metadata.

A concrete example

For **Finance/model.csv**, a result candidate might represent the **Revenue** row and **Current Period** cell with value **120**. Its object ID identifies the cell; its snapshot ID identifies the cited version; its stable file ID owns the usage history.

> **WHY THESE IDS MATTER**
>
> If model.csv becomes budget.csv, the cited old version still has its original path. Ranking resolves the same stable file identity to its current path for usage and relationships. A new file placed at model.csv has a different identity and receives none of that history.

Code references: [S1] rank; [S2] objectsAt and ensureIndex; [S6] timeline identity matching; [S7] persistence tables.

03 / IMPLEMENTATION GUIDE

The program around the ranker

The application is a local Node.js service with built-in SQLite and a content-addressed blob store. Browser and native macOS clients call its HTTP API. The index, terminal, editor and source picker read the same underlying source records.

1. **Capture and index**
   Files -> snapshots -> typed objects
2. **Retrieve eligible candidates**
   Query, claim, context, type and explicit scope
3. **Validate and classify**
   Project identity, confidence, tier and band
4. **Order and diversify**
   Relationship, capped usage and file rotation
5. **Display; record successful use**
   Explanation -> deliberate action -> local history

Who does each job?

TimelineService captures the directory and tracks immutable snapshots. **PredictiveProvenanceService** indexes those snapshots, detects claims and retrieves candidates. **RecommendationRanking** validates, personalizes, sorts and diversifies the candidate list.

DependencyGraph gives indexed objects terminal-friendly names and source addresses. **TerminalCommandExecutor** parses command positions and applies completion policies. **src/http/api.js** connects these services to the browser and native clients.

Where the feedback loop begins

Displaying a result is read-only. After a suggestion is successfully accepted or a source deliberately opens, the client or dependency-link transaction records local usage. A later ranking request reads that history. No recommendation itself writes a manuscript or publishes a share.

> **DETERMINISTIC BEHAVIOR**
>
> The same project state, candidate list, options and clock time produce the same rank order. Current suggestions use token matching, source metadata and explicit formulas. They do not require a language-model call or a learned ranking model.

Code references: [S2] service constructor, prediction and linking; [S4-S5] terminal; [S8-S10] clients and API.

04 / IMPLEMENTATION GUIDE

How search retrieves candidates

Retrieval creates the list that ranking may reorder. Popularity does not add a missing object to that list. An explicit search can be broad, but its candidates must still come from indexed project records and satisfy the requested scope.

Direct matching

The shared helper trims text, lowercases it and collapses whitespace. It first checks exact full paths, filenames, filename stems, labels and names. It then checks object IDs, variable names, code symbols, citation keys, DOI, PMID and structural identifiers, followed by exact literal values. A name prefix of at least two characters is also a protected direct match.

Literal values preserve signs and punctuation: **12** is different from **120** and **-12**. Scientific result matching, discussed next, has additional numeric-equivalence rules.

Source-service search signal	Initial relevance
Exact name, identifier, value or full alias	1.00
Shared name-prefix match	0.90
Other query content	Fraction of query terms occurring as substrings in indexed search text or path
Partial alias context	0.75 x fraction of query terms found in the alias
Matching extra context	Up to +0.03 for a non-direct query match, capped at 1
No typed query, matching supplied context	0.25 + 0.35 x context-term coverage
No typed query, no contextual hit	0.20 for browsing; stronger contextual hits have a higher tier

The terminal has a naming layer

Terminal search also checks slugged names, aliases and source filters. Name containment is 0.66 and content coverage is weighted by 0.60. File autocomplete uses basename prefixes at 0.94, full-path prefixes at 0.90 and containment at 0.68. An exact filename search scopes the terminal result to that file.

The services pre-sort genuine ties using their own source preferences before handing the list to the shared ranker. These differences in retrieval do not change the shared ranking priorities.

Code references: [S1] directRecommendationMatch; [S2] search; [S4] search, sourceSuggestions and nameSuggestions.

05 / IMPLEMENTATION GUIDE

How scientific results get a score

When the user writes a claim, Lasso detects its type and selects compatible object types. A numerical result looks for values and results; a method looks for code or protocols; a literature citation looks for papers or references. It scores several evidence signals rather than treating every matching number as interchangeable.

Signal, usually in [0,1]	Weight	What it captures
V: value match	0.56	Numeric values, signs, direction and supported percent/fold equivalence
E: entity overlap	0.20	Words and entity prefixes from the claim, sentence and section
X: variable similarity	0.48	Named variable, dictionary definition or inferred column profile
A: author similarity	0.16	Publication author match
Q: reference similarity	0.50	Citation metadata, author mode or title mode
C: component similarity	0.62	Figure or table identifier
Y: matching year	0.10	The publication year occurs in the claim
T: compatible type	0.09	The object is a preferred source type
G: recorded provenance	0.03	The source path participates in recorded provenance

```
R = min(1, .56V + .20E + .48X + .16A + .50Q
           + .62C + .10Y + .09T + .03G)
```

Corrections and eligibility

For an ENTITY claim, a DATASET_ROW with entity overlap >= 0.68 gets an additional 0.18, capped at 1. If a named claim variable conflicts with the candidate variable (similarity below 0.20), the score is multiplied by 0.42. The returned score is rounded to three decimals.

Match strength is a separate value: **max(V, E, X, Q, C)**. It measures the strongest evidence signal. It is used to silence weak automatic suggestions. The relevance score is a weighted heuristic, not a calibrated probability that the claim is correct.

Retrieval requires **R >= .08** for literature or **.20** otherwise. Scientific tier 0 accepts **V >= .80** and (no named claim variable or **X >= .70**); **Q >= .90**; an exact author match; **C >= .90**; **X >= .90**; or **E >= .90** for entity/method claims. Other retained candidates enter tier 1.

A reference query requires **Q >= .68** or **A >= .70**. An explicit publication year must agree with the recorded candidate year; missing year metadata is allowed. These rules prevent popularity from rescuing an incompatible citation.

Code references: [S2] candidatesForClaim, valueMatch and related similarity functions.

06 / IMPLEMENTATION GUIDE

Writing suggestions use source text

Writing has several retrieval paths. All return real indexed source identities, then use the shared ranker. The program chooses a mode from the caret context and accepted source scope.

1. Study-variable completion

A fragment such as **mean placental wei** is matched against variable names, dataset rows and dictionary definitions. Exact, prefix and token-prefix matches receive different base scores, with small bonuses for dictionaries and useful data profiles. A raw relevance of at least **0.68** is required. Repeated representations of the same normalized variable are collapsed after ranking.

2. Contextual workspace suggestions

Study or methods prose can retrieve a source-backed variable, result, method or parameter. It uses token overlap plus object-type and data-profile bonuses. The caret must be at the end, with enough text and a recognized project intent. The final confidence floor is **0.54**. Identifier-like columns and irrelevant row counts are penalized.

3. Sentence completion

Lasso builds a corpus of source sentences and extends an incomplete trailing sentence. An exact sentence prefix gets **0.99**. Otherwise, a contiguous alignment of the last two to six tokens uses deterministic stemming and scientific synonym groups.

```
R = .62 x alignment + .33 x content overlap
minimum R = .54; insertion length <= 260 characters
```

The fragment must have at least eight characters and two tokens. The trimmed continuation must have at least three characters, and the result needs a linkable indexed object. Identical passages from different files or different bibliography works remain separate candidates.

4. Continuing from a selected source

After a paper is accepted, an attribution cue such as **the authors found that** can restrict findings to that paper and its cited snapshot. After a project object is accepted, related values can be restricted to its variable and source. This source scope remains authoritative even if another file is more popular.

> **WHAT GETS INSERTED**
>
> Sentence and finding suggestions come from actual project passages. Finding clauses may undergo deterministic cleanup for grammar. Accepting text carries the source object and source snapshot needed to create a provenance link. Detailed variable and workspace formulas are in the appendix.

Code references: [S2] predictionCandidates and source-scoped retrieval; [S3] completeSentence and completeFromLinkedSource.

07 / IMPLEMENTATION GUIDE

The exact ranking decision order

The ranker first rejects invalid project/file/object identities and below-threshold candidates. It preserves the incoming raw score as **relevanceScore** and stores usage separately. For a nonempty effective query, sorting compares the following keys in order and stops at the first difference.

Order	Key	Preferred direction
1	Relevance tier	Lower is better
2	Direct-match specificity	Exact literal or domain-direct evidence (2) before literal name prefix (1)
3	Relevance band	Higher is better
4	Document relationship	Related before unrelated
5	Raw relevance + usage boost	Higher is better
6	Unmodified relevance	Higher is better
7	Original caller position	Earlier is better; retains the caller tie rules

Tiers and bands

Tier 0 is a protected direct match. **Tier 1** represents current query/context relevance. **Tier 2** can represent a relationship without positive relevance in the generic classifier. **Tier 3** represents available/background candidates. Callers can supply domain tiers; literal direct checks take priority. Most retrieved scientific candidates use tiers 0 or 1.

```
band = floor((relevanceScore + 1e-9) / 0.08)
[0.80, 0.88) -> band 10
[0.88, 0.96) -> band 11
```

The tiny epsilon stabilizes exact decimal boundaries in floating-point arithmetic. A strong scientific value, author, variable or component match can be tier 0 even when the entire query is not a literal string match. Caller-supplied tier 0 gets specificity 2 unless the shared matcher finds a literal name prefix, which gets 1.

No query and no supplied context

All candidates use tier 3. Sorting then compares **weighted recency, frequency, relationship, raw relevance and caller order**. When the caller has supplied selected text or sentence context, it passes that context as an effective query instead.

The code has final path/ID comparisons after caller position. Distinct list entries already have distinct positions, so genuine ties ordinarily preserve caller order.

Code references: [S1] rank comparator and tier preparation; [S2/S4] caller scoring and context forwarding.

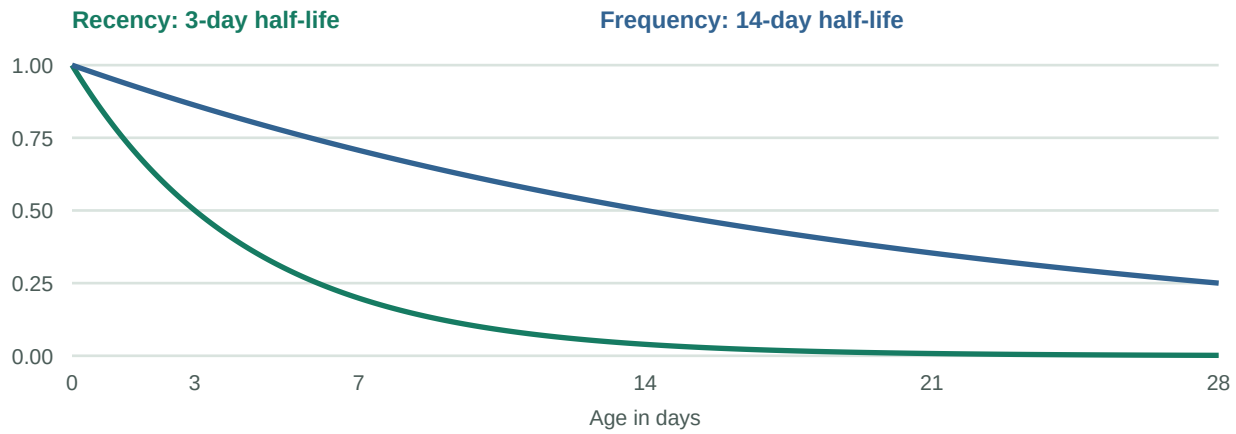
08 / IMPLEMENTATION GUIDE

The small, capped usage boost

Usage is aggregated by stable file identity. An accepted source has more weight than a deliberate open. Each row below is already deduplicated by project, file, action kind and session. Let **w** be its action weight, **a** its age since latest use and **b** its age since the first action in that session, measured in days.

Action	Weight
Successful acceptance / citation copy	1.00
Successful deliberate source open	0.35
Appearance, hover, navigation, automatic preview or failure	0.00; no usage record

```
recentScore R_u = max over rows [ w x 2^(-a/3) ]
decayedFrequency D = sum over rows [ w x 2^(-b/14) ]
frequencyScore F_u = 1 - exp(-D/5)
usageBoost U = min(.06, .04R_u + .02F_u)
```



The curves show the decay factor for one action before saturation or the final usage coefficients. Recency has a three-day half-life; frequency has a fourteen-day half-life. Negative ages are clamped to zero.

Numbers you can check

One fresh acceptance gives **R_u = 1**, **F_u = 0.181269** and **U = 0.043625**. One fresh open gives **R_u = 0.35**, **F_u = 0.067606** and **U = 0.015352**. Many separate recent sessions approach a maximum boost of **0.06**.

Recency is the **maximum weighted decayed action**, not a timestamp sort or sum. A one-day-old acceptance has recency about 0.794 and can outrank a brand-new open at 0.35. This preserves the higher weight of acceptance.

Code references: [S1] usageForProject; [S7] recommendation_usage schema.

09 / IMPLEMENTATION GUIDE

Sessions, relationships and renames

Two timestamps prevent click inflation

The usage primary key is **(project_id, project_file_id, kind, session_id)**. The first acceptance or open inserts a row. Repeating the same kind in the same session changes **last_used_at_ms** but leaves **occurred_at_ms** unchanged. Recency is refreshed; session frequency does not increase.

```
new file/action/session -> insert(first_time, last_time)
existing file/action/session -> update(last_time only)
new associated document -> add document association
```

A file can contribute one acceptance and one open per session, for at most **1.35** weighted frequency before decay. The browser uses a page-session ID and the native app a process-session ID. Without a client ID, the server uses a fixed thirty-minute clock bucket; this is not an inactivity timer.

A document relationship is a Boolean

A source is related if it is linked to the active document, supplied as a related source by the caller, or deliberately used with that document before. Current links use the latest dependency versions, excluding removed and broken links. Source paths are resolved through stable file identities.

Recorded document co-use persists without decay. Activity scores decay separately. Removing a link removes that link-based relationship, but a prior co-use association can still relate the same source to the document. A relationship has sort precedence within a band; it is not an extra number added to relevance.

Local persistence and identity checks

SQLite stores both usage and document associations. Usage survives restart. Source/object and dependency identities must belong to the requested project; when both are supplied, they must agree. An authoritative object ID can resolve a stale path after a rename. Invalid or foreign identities are rejected.

> **RENAME EXAMPLE**
>
> **File F12:** model.csv -> budget.csv, with its existing usage.
> **File F99:** a new model.csv, with no inherited usage.
> An old citation to F12 is still validated against its historical snapshot. Rename continuity depends on the timeline recognizing a unique inode/device or content-hash match.

Normal usage aggregation includes files present in the current snapshot. Historical source validation does not guarantee a deleted file will still receive a usage boost.

Code references: [S1] record, usageForProject, relatedFilePaths and rank; [S2] recordRecommendationUsage; [S6-S7] identity and storage.

10 / IMPLEMENTATION GUIDE

Diversity: rank objects, show choices

A spreadsheet may contain many equally relevant objects. Without diversity, one file could consume the result menu. Lasso first sorts all prepared candidates, then groups them by **tier, direct specificity and relevance band**. It rotates file identities within each group.

Already sorted in one group	Diversified selection
A1, A2, A3, A4, B1, C1	Round 1: A1, B1, C1
The remaining A candidates	Round 2: A2
A3 and A4	Omitted by the default two-per-file cap

The actual procedure

For each group, scan the remaining candidates in their ranked order. Take at most one per file key in that round. Repeat while slots remain. A global count across groups limits each file key to **two** selections by default. Groups retain the order established by the comparator.

File identity, rather than path spelling, groups ordinary objects. Historical and renamed representations of the same file do not gain extra slots. A caller can provide another diversity key when the item represents an independent choice.

Three important exceptions

One eligible source: if the entire pool has one diversity key, Lasso returns its ranked choices without the cap.
Explicit source scope: source-focused callers disable diversity, allowing the selected file or paper to expose its relevant objects.
Bibliography works: each reference object gets its own key, preserving independently citable works in one .bib or similar file.

Reference works have separate diversity keys, but they still share their containing file's usage history. Diversity and personalization are different concerns.

WHAT DIVERSITY GUARANTEES

Rotation does not cross a relevance tier or band. Direct matches remain ahead of weaker context matches. If caps exhaust the available eligible choices, the function can return fewer than the requested limit; it does not refill the menu with capped or ineligible objects.

Code references: [S1] rank diversification loop; [S2/S4] diversity keys and source-scope options; [S3] bibliography corpus identity.

11 / IMPLEMENTATION GUIDE

Policies for each UI surface

The ranking priorities are shared, but retrieval thresholds and output limits depend on whether the user is quietly typing or explicitly asking for sources. These policies run before popularity can affect order.

Surface	Eligibility / limit
Terminal file, name or address autocomplete	Typed fragment: raw relevance >= 0.80, at most 5 source choices. Empty fragment permits recent-file or contextual browsing.
Scientific typing predictions	At most 5 across all result groups. Direct evidence or non-direct match strength >= 0.40. Study-variable threshold >= 0.68.
Workspace / sentence completion	Original confidence >= 0.54, plus text, caret and source-identity checks. Sentence fallback produces one continuation.
Explicit prediction	surface=search allows up to 12. Keeps retrieval floors and source constraints but skips the automatic 0.40 / 0.68 final strength filter.
Explicit terminal completion	surface=search allows up to 14 source choices without the automatic 0.80 floor. Explicit Tab invocation forwards this mode.
Explicit source-service / terminal search	Default limits 30 / 25. Query, source filters and ambiguity handling remain authoritative.
Source library / selected source	Library ranks the full eligible matching inventory and groups it into file rows. Explicit source drilldowns preserve the source's relevant choices.

Prediction groups share a slot budget

The prediction wrapper allocates slots to source-scoped completions first, workspace completions next, then claims ordered by proximity to the caret. Each group uses ranking, but the program does not merge every mode into one global sorted list. A sentence continuation is a fallback when no claim candidates are available.

The UI preserves the server's decisions

The source workbench keeps server rank order instead of alphabetically sorting files. It keeps the selected source while it remains eligible and displays a short reason. Older query responses are ignored. Source preview, arrow movement and hover do not record use.

Ordinary command names, flags, durations and directory completions are grammar/prefix lists. They do not receive file-personalization boosts. Automatic source exclusions continue to apply; explicit search and chosen source scope remain available where the existing rules allow them.

Code references: [S2] predict and search; [S5] suggestionsOfKind; [S8-S10] UI mode, response and action handling.

12 / IMPLEMENTATION GUIDE

Worked examples: why a result wins

These examples start with eligible prepared candidates. Their raw scores come from the retrieval surface; they are not universal scores for every file. For clarity, each candidate has a different file key.

Candidate	Tier	Raw R	Band	Related	Usage U
A: exact value	0	.810	10	No	.000
B: popular context	1	.990	12	No	.060
C: document source	1	.820	10	Yes	.000
D: popular context	1	.870	10	No	.060
E: context	1	.850	10	No	.000
F: regularly accepted	1	.830	10	No	.05995

The resulting order is A, B, C, D, F, E

A wins by direct tier despite its lower raw score.
B leads the context candidates because its relevance band is higher.
C wins among band-10 context candidates because a relationship precedes usage.
D, **F** and **E** are unlinked in the same band, so totals .930, .88995 and .850 decide their order.

A modest acceptance can break a close tie

An unused source at **0.85** and an accepted source at **0.83** both lie in band 10. One fresh acceptance gives the second a boost of **0.043625**; its ordering total becomes **0.873625**, so it wins. Its displayed raw relevance remains **0.83**.

A band boundary blocks the boost

A source at **0.880** belongs to band 11. A popular source at **0.879** belongs to band 10. The 0.880 source wins even when the lower source approaches an ordering total of 0.939. The comparator never reaches usage for this pair.

Confidence filtering comes even earlier

If a typing surface requires 0.80, a candidate at 0.79 is removed before sorting. A 0.06 usage boost cannot rescue it. Similarly, invalid project identities, excluded paths and incompatible source types cannot gain eligibility through popularity.

Code references: [S1] comparator and usage calculation; [S11] bounded-usage and confidence regression tests.

13 / IMPLEMENTATION GUIDE

Follow one request through the code

Suppose a user types **Supplies were 120.** in Draft.md, and the project has several CSV files. This is the path from text to a source-backed recommendation.

Step	Execution
1. Request	The client POSTs text, cursor and document context to /api/projects/:id/dependencies/predict.
2. Detect and retrieve	predictionCandidates detects a numerical claim, selects compatible object types and reads the current indexed snapshot.
3. Score evidence	candidatesForClaim computes value, variable, entity and type evidence. It applies source scope, exclusions and scientific gates.
4. Prepare ranking	rank validates the candidate's project, snapshot, path and object version, resolves file identity and attaches local usage and relationship fields.
5. Sort and allocate	Candidates are compared by tier, specificity, band, relationship and bounded usage. Diversity and the typing slot budget reduce the list.
6. Display	The client shows source labels, values, locations and a reason. No usage is recorded for display or automatic preview.
7. Successful action	Acceptance links the document span to the chosen source snapshot. Human link/batch acceptance records usage in the same database transaction.

Conceptual pseudocode

```
pool = retrieve_from_project_index(query, context, scope)
pool = apply_source_rules_and_evidence_gates(pool)
for c in pool:
    validate_project_snapshot_file_and_object(c)
    c.R = original_retrieval_score(c)
    c.tier, c.specificity, c.band = classify(c)
    c.related = document_relationship(c)
    c.U = capped_usage_for_stable_file(c)
ordered = lexicographic_sort(pool)
shown = diversify_within_groups(ordered, surface_limit)
# On verified acceptance/open only:
record_usage(original_project, stable_source, session)
```

The real code may rank at retrieval and again at the prediction wrapper. This lets the wrapper enforce its final surface budget while retaining source-specific constraints and original confidence. Explicitly scoped source completion stays tied to its chosen source.

Code references: [S2] predictionCandidates, candidatesForClaim, predict and link/linkMany; [S1] rank; [S8] predict route.

14 / IMPLEMENTATION GUIDE

Debugging, performance and limits

A practical debugging sequence

1. Confirm the file/object is indexed in the requested project snapshot.
2. Check explicit source scope, source type and exclusions.
3. Inspect raw relevance and match strength before personalization.
4. Compare tier, specificity and band before considering relationships or usage.
5. Check the session ID and both action timestamps if usage seems inflated.
6. Inspect diversity keys and final group budgets when expected choices are absent.

Cost of the current implementation

For **n** prepared candidates, sorting costs **O(n log n)**. Preparation also reads usage rows, document associations and validated file/object versions, then performs token and metadata matching. These database and text-processing costs are additional.

Diversity uses array scans, splicing and repeated rounds. Its worst case is **O(n squared)**, although small display limits usually reduce the practical work. Snapshot caches avoid repeated extraction for unchanged data. The library skips per-object lineage enrichment for its full inventory; detailed inspection retrieves provenance separately.

Limits that affect recall

The scientific index caps descriptors at **10,000 per snapshot**, with per-table cell limits and file-size/type gates. The sentence corpus caps at **12,000 sentences**, accepts bounded sentence lengths and excludes fenced code. Not every tracked file or readable plain-text file is automatically a linkable scientific source. Missing candidates may reflect these retrieval/index limits rather than their rank.

What the regression suite establishes

Tests exercise priority order, capped boosts, confidence filtering, decay, session deduplication, project isolation, restart persistence, renames, reused old paths, historical identities, bibliography diversity, explicit scope, exclusions, passive versus deliberate actions and stale UI responses. At completion of the implementation, **188 Node tests**, **87 native checks** and a full macOS compile passed.

> **INTERPRETATION LIMITS**
>
> Scores are hand-tuned heuristics, not probabilities. Matching uses structured evidence and token/synonym rules. A correct-looking rank does not establish scientific truth. Provenance remains an explicit link to a real source version, and ambiguity resolution stays under the existing alias/source-selection rules.

Code references: [S1-S3] rank/index/corpus limits; [S4] resolve; [S11] tests. Verification counts refer to the implementation run, not a new PDF-specific build.

15 / IMPLEMENTATION GUIDE

A map of the implementation

Paths below are relative to the attached Lasso repository. The source labels used throughout this guide refer to these files. Start with S1 and S2, then follow the surface you want to understand.

Ref	Read this file / area	What to look for
S1	src/dependencies/ranking.js	directRecommendationMatch; record; usageForProject; relatedFilePaths; rank
S2	src/dependencies/service.js	ensureIndex; objectsAt; candidatesForClaim; predict; search; recordRecommendationUsage; link and linkMany
S3	src/dependencies/completion.js	Corpus construction; sentence alignment; linked-source findings
S4	src/terminal/graph.js	Named objects, search, aliases, source/address suggestions and ambiguity resolution
S5	src/terminal/executor.js	Completion grammar, suggestionsOfKind, typing versus explicit limits
S6	src/history/service.js and scanner.js	Project snapshots, stable file IDs, rename recognition and scanning rules
S7	src/db.js; migrations/014_recommendation_usage.sql	Local SQLite migration and usage/document-association schema
S8	src/server.js; src/http/api.js	Application service wiring and routes to prediction, search, library and usage
S9	public/app.js, source-workbench.js and dependency-shell.js	Display order, explanations, successful actions, session IDs and request races
S10	desktop/GlobalAssist.swift and AssistSourceIdentity.swift	Native suggestions, verified insertion/open behavior and captured project identity
S11	test/recommendation-ranking.test.js and recommendation-integration.test.js; terminal, source-workbench, sentence and desktop tests	Executable examples of ranking guarantees and cross-surface behavior
S12	docs/predictive-provenance-editor.md; README.md	Architecture and documented parameters

Keep four concepts separate

Eligibility: may this source participate?
Relevance: how well does its indexed evidence match?
Personalization: which close eligible source has useful local history?
Presentation: how do groups, slot budgets and file diversity produce a readable menu?

Code references: This guide describes the current local working tree, including its uncommitted implementation changes. No remote repository state is assumed.

16 / IMPLEMENTATION GUIDE

Appendix: specialized score formulas

The shared ranker accepts relevance produced by each retrieval path. These formulas explain the two writing paths whose bonuses are more specialized than ordinary search. They are tuning rules in source code, not learned parameters.

Study-variable retrieval

Component	Contribution
Base match (choose first applicable)	Exact .91; prefix .82; contains .74; all-query-word token-prefix .68; otherwise 0
Object and source bonuses	VARIABLE +.04; DATASET_ROW +.065; dictionary +.03; data-like path +.02
Profile quality (choose one)	Range +.025; samples +.012; dictionary +.01; otherwise -.015
Output	Cap at .99; round to 3 decimals; require >= .68. Rank before deduplicating normalized variable names.

Contextual workspace retrieval

```
R = .18 + .52 x token overlap + applicable bonuses
```

Condition	Adjustment
Result intent and compatible result type	RESULT +.32; VARIABLE +.28; other compatible data/result object +.16
Method intent and compatible method type	+.36
Variable with neither result nor method intent	+.20
Dictionary / data-like path	+.08 / +.06
Variable profile	Range +.08; samples +.035; deficient inferred profile -.06
Row-count metric without sample-size intent	-.32
Identifier/group/cohort-like variable	Multiply resulting score by .38
Output	Cap at .98; round to 3 decimals; initial retrieval floor .34, final ranking floor .54.

Workspace queries require the caret at the end, a recognized project intent, no literature intent, at least six sentence characters and two variable words. The applicable bonuses are determined by those intents and object metadata.

Linked-paper findings: confidence is **min(.99, .48 + .48 x finding relevance)**. With a fragment, finding relevance combines .72 x prefix match, .18 x containment and .34 x content overlap; without one it uses finding signals. Source identity and extractive quality checks still apply.

Code references: [S2] candidatesForStudyVariable and workspaceSuggestions; [S3] completeFromLinkedSource.